Package ‘taxogeno’

August 10, 2026

Title Comparative Phylogenetic Analysis of Complete Proteomes

Version 0.1.0

Description A comprehensive ETL (Extract-Transform-Load) system for processing, storing, and analyzing complete proteomes functionally annotated by Sma3s. The package builds a normalized PostgreSQL database with functional annotations (GO, GO Slim, Keywords, EC, Pathways), protein sequences, taxonomic relationships, and Euclidean distance matrices between proteomes. Includes a Shiny application for interactive exploration and visualization of phylogenetic relationships.

License MIT + file LICENSE

Encoding UTF-8

LazyData true

Roxygen list(markdown = TRUE)

Depends R (>= 4.0)

Imports DBI,
RPostgres,
uuid,
tools,
parallel

Suggests shiny,
shinyTree,
ggplot2,
ggiraph,
DT,
jsonlite,
knitr,
rmarkdown,
testthat

SystemRequirements PostgreSQL (>= 12) with pl/R and XML extensions,
Perl (for load_ncbi_taxonomy.pl),
OWLTools (for GO Slim mapping),
BioSQL schema,
wget (for external database downloads)

VignetteBuilder knitr

URL https://github.com/jmengom/taxogeno

BugReports https://github.com/jmengom/taxogeno/issues

Config/roxygen2/version 8.1.0

Contents

`calculateEuclideanDistances`

Calculate Euclidean Distances Between Proteomes

Description

Computes Euclidean distances between proteomes based on their normalized annotation vectors (GO Slim or Keywords). Uses sparse matrix representation for memory efficiency.

Usage

```
calculateEuclideanDistances(normalizedAnnotationDf)
```

Arguments

normalizedAnnotationDf	Data frame with columns: \itemproteomeidProteome identifier \itemannotkwidAnnotation term ID \itemannotkwrelvalNormalized annotation value (count/genecount)

Details

The function:

1. Builds a sparse matrix: proteomeid × annotkwid
2. Calculates Euclidean distance between rows (proteomes)
3. Converts the result to a data frame

Value

A data frame with distance information:

proteomeid_greatest	First proteome ID (greater numeric value)
proteomeid_least	Second proteome ID (lesser numeric value)
distance	Euclidean distance between the two proteomes

Examples

```
## Not run:
norm_df <- data.frame(
  proteomeid = c(1, 1, 2, 2),
  annotkwid = c("GO:0003674", "GO:0005575", "GO:0003674", "GO:0008150"),
  annotkwrelval = c(0.1, 0.05, 0.2, 0.15)
)
distances <- calculateEuclideanDistances(norm_df)

## End(Not run)
```

chunkVectorInEqualSizeFragments *Split Vector into Equal-Sized Fragments*

Description

Splits a vector into fragments of approximately equal size. This is used for memory-efficient processing when comparing proteomes in chunks.

Usage

```
chunkVectorInEqualSizeFragments(x, n)
```

Arguments

x	A vector to split.
n	Integer. The maximum size of each fragment.

Value

A list of vector fragments.

Examples

```
chunkVectorInEqualSizeFragments(1:100, n = 20)
```

convertGafToSummaryDf *Convert GAF Data Frame to Summary Data Frame*

Description

Aggregates a GAF data frame to count annotations per term and adds the proteome ID as a reference.

Usage

```
convertGafToSummaryDf(gafDf, proteomeId)
```

Arguments

gafDf	A GAF-format data frame.
proteomeId	Integer. The proteome ID to associate with the summary.

Value

A data frame with columns:

annotkwid	Annotation term ID
annotkwcount	Count of annotations for this term
proteomeid	The proteome ID

Examples

```
## Not run:
summary_df <- convertGafToSummaryDf(gaf_df, proteome_id = 1)

## End(Not run)
```

---

deleteProteome *Delete Proteomes from the Database*

Description

Deletes one or more proteomes and all associated data using the taxogeno.delete_proteome stored procedure.

Usage

```
deleteProteome(dbConn, proteomeIdVec)
```

Arguments

dbConn	A DBI database connection object.
proteomeIdVec	Integer vector. Proteome IDs to delete.

Details

The deletion is performed via a PostgreSQL stored procedure that handles cascade deletion of all related records including:

- GO Slim summaries
- Distance matrices
- Gene records
- All annotation relationships (GO, Keywords, EC, Pathways)

Value

NULL (invisible). The function deletes data from the database.

Examples

```
## Not run:
conn <- dbConnect(RPostgres::Postgres(), dbname = "taxogeno")
deleteProteome(conn, c(1, 2, 3))

## End(Not run)
```

`distAsDf` *Convert Distance Object to Data Frame*

Description

Converts an R `dist` object to a data frame with rows, columns, and values.

Usage

```
distAsDf(inDist)
```

Arguments

`inDist`	A `dist` object.

Value

A data frame with three columns:

`row`	Row label
`col`	Column label
`value`	Distance value

Examples

```
d <- dist(matrix(rnorm(100), nrow = 10))
df <- distAsDf(d)
head(df)
```

`extractColumnDfFromInsertedGeneAnnotationDf` *Extract Semicolon-Separated Column into 1:N Relationship*

Description

Converts a column containing semicolon-separated values into a normalized 1:N relationship data frame. Each unique value gets its own row linked to the original gene ID.

Usage

```
extractColumnDfFromInsertedGeneAnnotationDf(
  insertedGeneAnnotationDf,
  colName,
  newColName = colName
)
```

Arguments

`insertedGeneAnnotationDf`	Data frame containing gene annotations. Must have a "geneid" column.
`colName`	Character string. Name of the column to expand.
`newColName`	Character string. Name for the expanded column. Defaults to `colName`.

Value

A data frame with two columns:

`geneid`	The gene identifier (repeated for each value)
`newColName`	The extracted values (one per row)

Examples

```
## Not run:
# Input: geneid | enzyme
#        1      | EC1;EC2;EC3
# Output: geneid | ec
#         1      | EC1
#         1      | EC2
#         1      | EC3
enzyme_df <- extractColumnDfFromInsertedGeneAnnotationDf(annot_df, "enzyme", "ec")

## End(Not run)
```

`generateGafDf` *Generate Gene Association Format (GAF) Data Frame*

Description

Creates a GAF data frame from ontology annotations, following the Gene Ontology Consortium's GAF 2.0 specification.

Usage

```
generateGafDf(dbConn, ontologyAnnotDf)
```

Arguments

`dbConn`	A DBI database connection object (used for future extensions).
`ontologyAnnotDf`	Data frame with columns: \itemgeneidGene identifier \itemannotkwidAnnotation term ID \itemaspectGO aspect (C, F, or P)

Value

A data frame in GAF 2.0 format with 17 columns:

`db`	Database source ("taxogeno")
`dbobjectid`	Gene ID
`dbobjectsymbol`	Gene symbol (same as ID)
`qualifier`	Qualifier (NA)
`annotkwid`	GO term ID
`dbreference`	Reference (NA)
`evidencecode`	Evidence code ("IEA")
`with`	With/from (NA)
`aspect`	GO aspect
`dbobjectname`	Object name (NA)
`dbobjectsynonym`	Synonyms (NA)
`dbobjecttype`	Object type ("protein")
`taxon`	Taxon (NA)
`date`	Current timestamp in ISO format
`assignedby`	Source ("sma3s")
`annotationextension`	Extensions (NA)
`geneproductformid`	Gene product form ID (NA)

Examples

```
## Not run:
annot_df <- data.frame(
  geneid = c(1, 2),
  annotkwid = c("GO:0003674", "GO:0005575"),
  aspect = c("F", "C")
)
gaf_df <- generateGafDf(conn, annot_df)

## End(Not run)
```

`getGeneratedGoslimNormalizedAnnotationForProteomeIdVec`
Get Generated GO Slim Normalized Annotation for Proteomes

Description

Retrieves normalized GO Slim annotation vectors for a set of proteomes from the generated_goslim_summary table.

Usage

```
getGeneratedGoslimNormalizedAnnotationForProteomeIdVec(dbConn, proteomeIdVec)
```

Arguments

`dbConn`	A DBI database connection object.
`proteomeIdVec`	Integer vector. Proteome IDs to retrieve.

Value

A data frame with columns:

`proteomeid`	Proteome identifier
`annotkwid`	GO Slim term ID
`annotkwcount`	Raw annotation count
`annotkwrelval`	Normalized value (count/genecount)

Examples

```
## Not run:
norm_df <- getGeneratedGoslimNormalizedAnnotationForProteomeIdVec(conn, c(1, 2, 3))

## End(Not run)
```

getKeywordNormalizedAnnotationForProteomeIdVec

Get Keyword Normalized Annotation for Proteomes

Description

Retrieves normalized keyword annotation vectors for a set of proteomes. Uses a Cartesian product of proteomes and all available keywords.

Usage

```
getKeywordNormalizedAnnotationForProteomeIdVec(dbConn, proteomeIdVec)
```

Arguments

dbConn	A DBI database connection object.
proteomeIdVec	Integer vector. Proteome IDs to retrieve.

Value

A data frame with columns:

proteomeid	Proteome identifier
annotkwid	Keyword ID
annotkwrelval	Normalized value (count/genecount)

Examples

```
## Not run:
norm_df <- getKeywordNormalizedAnnotationForProteomeIdVec(conn, c(1, 2, 3))

## End(Not run)
```

getLastProteomeId

Get the Last (Maximum) Proteome ID

Description

Retrieves the highest proteome ID from the proteome table.

Usage

```
getLastProteomeId(dbConn)
```

Arguments

dbConn	A DBI database connection object.

Value

Integer. The maximum proteome ID, or NA if no proteomes exist.

Examples

```
## Not run:
conn <- dbConnect(RPostgres::Postgres(), dbname = "taxogeno")
last_id <- getLastProteomeId(conn)

## End(Not run)
```

---

getNormalizedAnnotationForProteomeIdVec

Get Normalized Annotation for Proteomes

Description

Retrieves normalized annotation vectors (GO Slim or Keywords) for a set of proteomes from the database. The normalized value is calculated as count / genecount.

Usage

```
getNormalizedAnnotationForProteomeIdVec(dbConn, annotationType, proteomeIdVec)
```

Arguments

dbConn	A DBI database connection object.
annotationType	Character string. Must be "generated_goslim" or "keyword".
proteomeIdVec	Integer vector. Proteome IDs to retrieve.

Value

A data frame with columns:

proteomeid	Proteome identifier
annotkwid	Annotation term ID
annotkwcount	Raw annotation count
annotkwrelval	Normalized value (count/genecount)

Examples

```
## Not run:
norm_df <- getNormalizedAnnotationForProteomeIdVec(
  conn,
  "generated_goslim",
  c(1, 2, 3)
)

## End(Not run)
```

`initializeTaxogeno` *Initialize Taxogeno Database with External Reference Data*

Description

Performs a complete initialization of the taxogeno database by downloading and installing all external reference datasets required for the system to function properly. This is a one-time setup step that must be completed before using the taxogeno package for proteome analysis.

Usage

```
initializeTaxogeno(dbConn)
```

Arguments

`dbConn`	A DBI database connection object to the taxogeno database.

Details

This function sequentially updates four essential external data sources:

1. **NCBI Assembly Summary GenBank**: Downloads the complete assembly summary from NCBI GenBank containing metadata for all publicly available genome assemblies. This is stored in the `ncbi.assembly_summary_genbank` table and is used for mapping GCA accessions to taxonomic and assembly information.
2. **NCBI Taxonomy (BioSQL)**: Downloads the complete NCBI taxonomy dump and loads it into the `biosql` schema using the BioSQL taxonomy loading script. This provides the full taxonomic hierarchy used for phylogenetic analysis and ancestor/descendant queries.
3. **UniProt Keywords**: Downloads both OBO and TSV formats of UniProt keywords, which are used for functional annotation of proteins. These are stored in the `uniprot.uniprot_keyword` and `uniprot.uniprot_keyword_category` tables.
4. **Gene Ontology (GO)**: Downloads the complete GO ontology in OWL format along with the GO Slim generic subset. These are parsed using PostgreSQL's XML capabilities and stored in the `gene_ontology` schema for annotation mapping and enrichment analysis.

Value

NULL (invisible). The function updates the database directly.

Dependencies

System Requirements:

- `wget`: Required for downloading files from FTP/HTTP servers
- `Perl`: Required for running the BioSQL taxonomy loading script
- `PostgreSQL` with `XML` extension enabled

R Packages (already imported):

- `DBI`: For database operations
- `RPostgres`: For PostgreSQL connectivity

Time and Resources

The initialization process is resource-intensive:

- **Download size**: Approximately 350-400 MB total
- **Database size**: Approximately 500-700 MB after insertion
- **Time**: 10-30 minutes depending on network speed and system performance

Error Handling

If any of the four updates fails:

- The function will stop execution with an error message
- The database may be left in a partially updated state
- You can rerun the function to complete the failed update
- Check network connectivity and database permissions if errors occur

Usage Recommendations

- Run this function once during initial system setup
- Run monthly to keep reference data current with NCBI/GO releases
- Use with `withProgress` for progress tracking in Shiny apps

References

- NCBI Assembly: `ftp://ftp.ncbi.nlm.nih.gov/genomes/genbank/`
- NCBI Taxonomy: `ftp://ftp.ncbi.nih.gov/pub/taxonomy/`
- UniProt Keywords: `http://www.uniprot.org/keywords/`
- Gene Ontology: `http://current.geneontology.org/ontology/`

See Also

updateNcbiAssemblySummaryGenbank for NCBI assembly updates updateBiosqlNcbiTaxonomy for taxonomy updates updateUniprotKeyword for UniProt keyword updates updateGeneOntology for GO updates saveSma3sFileSet for loading proteome data after initialization

Examples

```
## Not run:
# Connect to the database
library(DBI)
library(RPostgres)

conn <- dbConnect(RPostgres::Postgres(),
                  dbname = "taxogeno",
                  host = "localhost",
                  port = 5432,
                  user = "postgres",
                  password = "password")

# Initialize the database (takes 10-30 minutes)
initializeTaxogeno(conn)

# Verify the installation
dbGetQuery(conn, "SELECT COUNT(*) FROM ncbi.assembly_summary_genbank")
dbGetQuery(conn, "SELECT COUNT(*) FROM biosql.taxon")
dbGetQuery(conn, "SELECT COUNT(*) FROM uniprot.uniprot_keyword")
dbGetQuery(conn, "SELECT COUNT(*) FROM gene_ontology.gene_ontology")

# Disconnect when done
dbDisconnect(conn)

## End(Not run)
```

---

ncbiAssemblyInfoFromGcaIdContainingString

Retrieve NCBI Assembly Information from GCA ID

Description

Queries the NCBI assembly summary database for information about an assembly identified by a GCA accession number.

Usage

```
ncbiAssemblyInfoFromGcaIdContainingString(dbConn, gcaIdContainingString)
```

Arguments

dbConn	A DBI database connection object.
gcaIdContainingString	Character string containing a GCA ID (e.g., "GCA_000001405.28_GRCh38.p14"). The function extracts the GCA ID using regex.

Value

A list containing assembly information from the NCBI summary table:

assembly_accession	GCA accession
bioproject	BioProject ID
biosample	BioSample ID
wgs_master	WGS master accession
refseq_category	RefSeq category
taxid	NCBI taxonomy ID
species_taxid	Species taxonomy ID
organism_name	Scientific name
infraspecific_name	Strain/isolate name
isolate	Isolate identifier
version_status	Assembly version status
assembly_level	Assembly level (Complete, Chromosome, Scaffold, Contig)
release_type	Release type
genome_rep	Genome representation
date	Release date
asm_name	Assembly name
submitter	Submitting organization
gbrs_paired_asm	GenBank/RefSeq paired assembly
paired_asm_comp	Paired assembly comparison
ftp_path	FTP path for data download
excluded_from_refseq	Excluded from RefSeq flag
relation_to_type_material	Type material relationship

Examples

```
## Not run:
conn <- dbConnect(RPostgres::Postgres(), dbname = "taxogeno")
info <- ncbiAssemblyInfoFromGcaIdContainingString(conn, "GCA_000001405.28")
print(info$organism_name)

## End(Not run)
```

`owltoolsMap2SlimGafDf` *Map GO Terms to GO Slim Using OWLTools*

Description

Runs the OWLTools map2slim command to map specific GO terms to generic GO Slim terms. This is a crucial step for creating a simplified, high-level view of functional annotations.

Usage

```
owltoolsMap2SlimGafDf(gafDf, owl, subset = NA)
```

Arguments

`gafDf`	A GAF-format data frame (from `generateGafDf`).
`owl`	Character string. Path to the OWL ontology file.
`subset`	Character string. Optional subset name (e.g., "goslim_generic").

Details

The function:

1. Writes the GAF data to a temporary file
2. Executes OWLTools with the map2slim command
3. Reads the output from the temporary file
4. Cleans up temporary files

Value

A GAF-format data frame with mapped GO Slim terms.

Examples

```
## Not run:
mapped_gaf <- owltoolsMap2SlimGafDf(
  gaf_df,
  owl = "goslim_generic.owl",
  subset = "goslim_generic"
)

## End(Not run)
```

`readMultifasta` *Read Multi-FASTA File Line by Line*

Description

Reads a FASTA file containing multiple protein sequences using a streaming approach to avoid loading the entire file into memory. This is essential for processing large proteome files that may be several gigabytes.

Usage

```
readMultifasta(multifastaPath)
```

Arguments

`multifastaPath`	Character string. Path to the FASTA file.

Details

The function processes the file line by line using a state machine:

1. Looks for lines starting with ">" (FASTA header)
2. Collects subsequent lines as sequence until the next header
3. Handles sequences that span multiple lines

Value

A data frame with two columns:

`fastaheader`	The FASTA header (identifier after ">")
`aasequence`	The amino acid sequence (concatenated from all lines)

Examples

```
## Not run:
fasta_df <- readMultifasta("path/to/proteome.faa")
nrow(fasta_df)  # Number of protein sequences

## End(Not run)
```

`readSma3sAnnotation` *Read Sma3s Annotation TSV File*

Description

Reads the tab-separated annotation file generated by the Sma3s program. The file contains functional annotations for each protein sequence in a proteome.

Usage

```
readSma3sAnnotation(sma3sAnnotationPath)
```

Arguments

`sma3sAnnotationPath`	Character string. Path to the Sma3s TSV annotation file.

Value

A data frame with 14 columns:

`fastaheader`	Full FASTA header from the sequence file
`genename`	Gene name from the annotation
`genedescription`	Functional description of the gene product
`enzyme`	EC numbers (semicolon-separated)
`goc`	Gene Ontology Cellular Component IDs (semicolon-separated)
`gocname`	GO Cellular Component names
`gof`	Gene Ontology Molecular Function IDs (semicolon-separated)
`gofname`	GO Molecular Function names
`gop`	Gene Ontology Biological Process IDs (semicolon-separated)
`gopname`	GO Biological Process names
`keyword`	UniProt keywords (semicolon-separated)
`pathway`	Pathway annotations (semicolon-separated)
`goslim`	GO Slim terms (semicolon-separated)
`fastashortheader`	Short FASTA header (first word after ">")

Examples

```
## Not run:
annot_df <- readSma3sAnnotation("path/to/annotation.tsv")
head(annot_df)

## End(Not run)
```

`saveEuclideanDistancesForProteomeId` *Save Euclidean Distances for a Proteome*

Description

Calculates and caches Euclidean distances between a specific proteome and all other proteomes in the database for the specified annotation types. Uses chunking for memory-efficient processing.

Usage

```
saveEuclideanDistancesForProteomeId(dbConn, proteomeId, annotationTypeVec)
```

Arguments

`dbConn`	A DBI database connection object.
`proteomeId`	Integer. The proteome ID to compare.
`annotationTypeVec`	Character vector. Types of annotations to use: "generated_goslim" and/or "keyword".

Details

The function:

1. Validates the input parameters
2. Identifies proteomes not yet compared with the target
3. Processes comparisons in chunks to manage memory
4. Writes results to `euclidean_distances_generated_goslim` and/or `euclidean_distances_keyword`

Value

NULL (invisible). The function writes distances to database tables.

Examples

```
## Not run:
conn <- dbConnect(RPostgres::Postgres(), dbname = "taxogeno")
saveEuclideanDistancesForProteomeId(conn, 1, c("generated_goslim", "keyword"))

## End(Not run)
```

`saveGeneratedGoslimEuclideanDistancesForProteomeId`
Save GO Slim-Based Euclidean Distances for a Proteome

Description

Calculates and caches Euclidean distances based on GO Slim annotations between a specific proteome and all other proteomes in the database.

Usage

```
saveGeneratedGoslimEuclideanDistancesForProteomeId(dbConn, proteomeId)
```

Arguments

`dbConn`	A DBI database connection object.
`proteomeId`	Integer. The proteome ID to compare.

Details

This function is a wrapper around the more general `saveEuclideanDistancesForProteomeId` specifically for GO Slim annotations. It performs chunked processing to manage memory.

Value

NULL (invisible). The function writes distances to `taxogeno.euclidean_distances_generated_goslim`.

Examples

```
## Not run:
conn <- dbConnect(RPostgres::Postgres(), dbname = "taxogeno")
saveGeneratedGoslimEuclideanDistancesForProteomeId(conn, 1)

## End(Not run)
```

`saveGeneratedGoslimSummaryForGoStrictAnnotationData` *Save GO Slim Summary for Strict GO Annotations*

Description

Generates a GO Slim summary from strict GO annotations (C, F, P aspects) and saves it to the database. This is the main function for creating the simplified GO Slim representation of a proteome.

Usage

```
saveGeneratedGoslimSummaryForGoStrictAnnotationData(
  dbConn,
  proteomeId,
  goStrictDf
)
```

Arguments

`dbConn`	A DBI database connection object.
`proteomeId`	Integer. The proteome ID.
`goStrictDf`	Data frame with strict GO annotations: \itemgeneidGene identifier \itemannotkwidGO term ID \itemaspectGO aspect (C, F, or P)

Details

The function:

1. Generates a GAF data frame from the strict GO annotations
2. Maps the GAF to GO Slim using OWLTools
3. Converts the mapped GAF to a summary data frame
4. Writes the summary to `taxogeno.generated_goslim_summary`

Value

NULL (invisible). The function writes to the database.

Examples

```
## Not run:
conn <- dbConnect(RPostgres::Postgres(), dbname = "taxogeno")
saveGeneratedGoslimSummaryForGoStrictAnnotationData(conn, 1, go_df)

## End(Not run)
```

saveKeywordEuclideanDistancesForProteomeId *Save Keyword-Based Euclidean Distances for a Proteome*

Description

Calculates and caches Euclidean distances based on keyword annotations between a specific proteome and all other proteomes in the database.

Usage

```
saveKeywordEuclideanDistancesForProteomeId(dbConn, proteomeId)
```

Arguments

dbConn	A DBI database connection object.
proteomeId	Integer. The proteome ID to compare.

Details

This function is a wrapper around the more general saveEuclideanDistancesForProteomeId specifically for keyword annotations. It performs chunked processing to manage memory.

Value

NULL (invisible). The function writes distances to taxogeno.euclidean_distances_keyword.

Examples

```
## Not run:
conn <- dbConnect(RPostgres::Postgres(), dbname = "taxogeno")
saveKeywordEuclideanDistancesForProteomeId(conn, 1)

## End(Not run)
```

saveSma3sFileSet *Save Sma3s File Set to Database (Main ETL Pipeline)*

Description

The main ETL (Extract-Transform-Load) function that processes a complete Sma3s annotation set and loads it into the taxogeno database.

Usage

```
saveSma3sFileSet(
  dbConn,
  sma3sAnnotationFilePath,
  multifastaFilePath,
  tagVec = character(),
  isUserProteome = FALSE,
  gcaId = NA,
  ncbiTaxId = NA,
  dbName = NA,
  sourceUrl = NA,
  doUpdateNcbiAssemblyInfo = TRUE,
  webFile = FALSE,
  webSma3sAnnotationFileName = NA
)
```

Arguments

`dbConn`	A DBI database connection object.
`sma3sAnnotationFilePath`	Character string. Path to the Sma3s TSV annotation file.
`multifastaFilePath`	Character string. Path to the multi-FASTA protein file.
`tagVec`	Character vector. Optional tags to associate with the proteome.
`isUserProteome`	Logical. Whether this is a user-uploaded proteome.
`gcaId`	Character string. Optional GCA accession number.
`ncbiTaxId`	Integer. Optional NCBI taxonomy ID.
`dbName`	Character string. Source database name.
`sourceUrl`	Character string. Source URL for the data.
`doUpdateNcbiAssemblyInfo`	Logical. Whether to update NCBI assembly info.
`webFile`	Logical. Whether the files were uploaded via web interface. If TRUE, the function uses `webSma3sAnnotationFileName` for the basename.
`webSma3sAnnotationFileName`	Character string. Original filename for web uploads. Only used when `webFile = TRUE`.

Details

The pipeline performs these steps in order:

1. **Read annotations**: Loads the Sma3s annotation TSV file using `readSma3sAnnotation`.
2. **Read multifasta**: Loads the protein FASTA file using `readMultifasta`.
3. **Insert proteome metadata**: Creates a record in `taxogeno.proteome` with:
   - Basename (derived from file path or web filename)

- MD5 checksums for both input files
- Gene count
- User/assembly flags
- GCA ID and NCBI taxonomy ID (if provided)
- Source database name and URL
- Auto-generated jobid (UUID) for web tracking

4. **Update NCBI assembly info**: If `!isUserProteome && doUpdateNcbiAssemblyInfo`, extracts the GCA ID from the basename and updates the assembly information.
5. **Insert taxonomy**: If not a user proteome, inserts the taxon and its ancestors into the taxonomy table using `insert_taxon()`.
6. **Insert genes**: Stores basic gene information (fastaheader, fastashortheader, genename, genedescription) and links sequences from the multifasta.
7. **Insert annotation relationships**: For each gene, expands semicolon-separated fields into 1:N relationships and inserts into:
   - `gene_enzyme_rel` (EC numbers)
   - `gene_keyword_rel` (UniProt keywords)
   - `gene_pathway_rel` (Pathway annotations)
   - `gene_goslim_rel` (GO Slim terms)
   - `gene_goc_rel` (Cellular Component GO terms)
   - `gene_gof_rel` (Molecular Function GO terms)
   - `gene_gop_rel` (Biological Process GO terms)
8. **Generate GO Slim summaries**: Recalculates GO Slim annotations using OWLTools map2slim and saves aggregated counts to `generated_goslim_summary`.
9. **Calculate Euclidean distances**: Computes and caches distance matrices based on GO Slim and Keyword annotations via `saveEuclideanDistancesForProteomeId`.

Value

Character string. The generated jobid (UUID) for tracking the upload.

Job ID (UUID)

The function generates a UUID using `UUIDgenerate()` which is stored in the `proteome.jobid` column. This jobid is returned by the function and can be used to track the upload status in the web application.

Memory Management

The function handles large files efficiently:

- FASTA files are read line-by-line using streaming (see `readMultifasta`)
- Distance calculations use sparse matrix representation and chunking

Web Upload Support

When `webFile = TRUE`, the function:

- Uses `webSma3sAnnotationFileName` for the basename
- Still reads from `sma3sAnnotationFilePath` for the actual file content
- The jobid can be used to locate the uploaded files for cleanup

See Also

`readSma3sAnnotation` for reading the annotation file `readMultifasta` for reading FASTA files `updateNcbiAssemblyInfoForProteomeIdVec` for updating assembly metadata `extractColumnDfFromInsertedGeneAnno` for parsing semicolon-separated fields `saveGeneratedGoslimSummaryForGoStrictAnnotationData` for GO Slim processing `saveEuclideanDistancesForProteomeId` for distance calculations

Examples

```
## Not run:
conn <- dbConnect(RPostgres::Postgres(), dbname = "taxogeno")

# Standard usage for public proteomes
jobid <- saveSma3sFileSet(
  conn,
  "path/to/annotation.tsv",
  "path/to/proteome.faa",
  tagVec = c("bacteria", "example"),
  isUserProteome = FALSE,
  doUpdateNcbiAssemblyInfo = TRUE
)

# Web upload usage
jobid <- saveSma3sFileSet(
  conn,
  "/tmp/upload_1234.tsv",
  "/tmp/upload_1234.faa",
  tagVec = "user_upload",
  isUserProteome = TRUE,
  webFile = TRUE,
  webSma3sAnnotationFileName = "my_proteome.tsv"
)

print(paste("Upload job ID:", jobid))

## End(Not run)
```

`updateBiosqlNcbiTaxonomy` *Update BioSQL NCBI Taxonomy*

Description

Downloads the latest NCBI taxonomy dump and loads it into the biosql schema using the `load_ncbi_taxonomy.pl` Perl script.

Usage

```
updateBiosqlNcbiTaxonomy(dbConn)
```

Arguments

`dbConn`	A DBI database connection object.

Details

The function:

1. Downloads `taxdump.tar.gz` from NCBI FTP
2. Extracts the archive
3. Executes the Perl script `load_ncbi_taxonomy.pl`
4. Cleans up temporary files

Value

NULL (invisible). The function updates the database directly.

Examples

```
## Not run:
conn <- dbConnect(RPostgres::Postgres(), dbname = "taxogeno")
updateBiosqlNcbiTaxonomy(conn)

## End(Not run)
```

`updateGeneOntology` *Update Gene Ontology Tables*

Description

Downloads the latest Gene Ontology OWL files and parses them into the database using PostgreSQL's XMLTABLE functionality.

Usage

```
updateGeneOntology(dbConn)
```

Arguments

`dbConn`	A DBI database connection object.

Details

The function:

1. Downloads `go.owl` from Gene Ontology
2. Downloads `gislim_generic.owl` from Gene Ontology
3. Inserts OWL files into `gene_ontology.gene_ontology_xml`
4. Parses OWL XML using `XMLTABLE` to extract:
   - GO ID (`goid`)
   - GO Label (`golabel`)
   - GO Aspect (`goaspect`)
5. Updates `gene_ontology.gene_ontology`
6. Updates `gene_ontology.goslim_generic`

The parsing uses XMLTABLE with appropriate XML namespaces:

- `rdf`: `http://www.w3.org/1999/02/22-rdf-syntax-ns#`
- `owl`: `http://www.w3.org/2002/07/owl#`
- `oboInOwl`: `http://www.geneontology.org/formats/oboInOwl#`
- `rdfs`: `http://www.w3.org/2000/01/rdf-schema#`

Value

NULL (invisible). The function updates the database directly.

Examples

```
## Not run:
conn <- dbConnect(RPostgres::Postgres(), dbname = "taxogeno")
updateGeneOntology(conn)

## End(Not run)
```

`updateNcbiAssemblyInfoForProteomeIdVec`
Update NCBI Assembly Information for Proteomes

Description

Updates NCBI assembly information (GCA, taxonomy ID, source URL) for a set of proteomes by querying the NCBI assembly summary database.

Usage

```
updateNcbiAssemblyInfoForProteomeIdVec(dbConn, proteomeIdVec)
```

Arguments

`dbConn`	A DBI database connection object.
`proteomeIdVec`	Integer vector. Proteome IDs to update.

Details

For each proteome:

1. Retrieves existing GCA ID or uses basename as fallback
2. Queries NCBI assembly summary for updated information
3. Updates taxonomy if taxon has changed
4. Inserts new taxon if it doesn't exist
5. Updates proteome record with new assembly info

Value

NULL (invisible). The function updates the database directly.

Examples

```
## Not run:
conn <- dbConnect(RPostgres::Postgres(), dbname = "taxogeno")
updateNcbiAssemblyInfoForProteomeIdVec(conn, c(1, 2, 3))

## End(Not run)
```

`updateNcbiAssemblySummaryGenbank` *Update NCBI Assembly Summary GenBank Table*

Description

Downloads the latest NCBI assembly summary from GenBank and updates the database table `ncbi.assembly_summary_genbank`.

Usage

```
updateNcbiAssemblySummaryGenbank(dbConn)
```

Arguments

`dbConn`	A DBI database connection object.

Details

The function:

1. Downloads `assembly_summary_genbank.txt` from NCBI FTP
2. Reads the tab-separated file
3. Deletes the existing table contents
4. Inserts the new data

Value

NULL (invisible). The function updates the database directly.

Examples

```
## Not run:
conn <- dbConnect(RPostgres::Postgres(), dbname = "taxogeno")
updateNcbiAssemblySummaryGenbank(conn)

## End(Not run)
```

updateProteomeFieldInDB *Update a Proteome Field in the Database*

Description

Updates a specific field in the proteome table for a single proteome.

Usage

```
updateProteomeFieldInDB(dbConn, proteomeId, fieldName, newValue)
```

Arguments

dbConn	A DBI database connection object.
proteomeId	Integer. The proteome ID to update.
fieldName	Character string. The field name to update. Allowed values: "is_userproteome", "gcaid", "ncbitaxid", "scientific_name".
newValue	The new value for the field.

Value

NULL (invisible). The function updates the database directly.

Examples

```
## Not run:
conn <- dbConnect(RPostgres::Postgres(), dbname = "taxogeno")
updateProteomeFieldInDB(conn, 1, "is_userproteome", TRUE)

## End(Not run)
```

updateUniprotKeyword *Update UniProt Keywords*

Description

Downloads the latest UniProt keywords in both OBO and TSV formats and updates the database tables.

Usage

```
updateUniprotKeyword(dbConn)
```

Arguments

dbConn	A DBI database connection object.

Details

The function:

1. Downloads `keyword.obo` from UniProt
2. Downloads `keyword.tsv` from UniProt
3. Reads the TSV file
4. Updates `uniprot.uniprot_keyword_category`
5. Updates `uniprot.uniprot_keyword`

Value

NULL (invisible). The function updates the database directly.

Examples

```
## Not run:
conn <- dbConnect(RPostgres::Postgres(), dbname = "taxogeno")
updateUniprotKeyword(conn)

## End(Not run)
```

Index